

# VAT ENTERPRISE PLATFORM

## Phase 4: FastAPI Controllers, Dependency Injection & WebSockets Walkthrough

Carrier-Grade Multi-Vendor AI Diagnostic, Real-Time WebSockets & 4-Stage Remediation Engine

| DOCUMENT CLASSIFICATION       | ROLE / AUTHOR                 | TARGET PLATFORMS                     | ENGINEERING STATUS         |
|-------------------------------|-------------------------------|--------------------------------------|----------------------------|
| CONFIDENTIAL • TIER-1 CARRIER | Principal Solutions Architect | Cisco • Juniper • VeloCloud • Arista | PHASE 1–4 VERIFIED (57/57) |

### 1. Executive Summary & Core Architectural Tenets

The **Vendor-Aware AI Troubleshooter (VAT)** is engineered to diagnose and remediate mission-critical telecom and carrier network incidents in under 3 minutes. By enforcing **Clean Architecture / Hexagonal Architecture**, the platform guarantees zero framework coupling, air-gapped resilience, and strict adherence to three architectural tenets:

|                                                                                                                                                                                                   |                                                                                                                                                                                         |                                                                                                                                                                                                |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1. Deterministic Grounding</b><br>Zero speculative hallucination. Every diagnosis, pre-check, CLI command, and rollback procedure is strictly grounded in indexed official vendor TAC manuals. | <b>2. Air-Gapped Degradation</b><br>The system seamlessly falls back to an in-memory multi-vendor corpus if PostgreSQL or cloud LLM APIs are unreachable, ensuring 100% offline uptime. | <b>3. 4-Stage Safe Runbooks</b><br>Enforces sequential safety: Read-only pre-checks before any configuration mutations, followed by convergence verification and automated rollback playbooks. |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### 2. Clean Architecture Layering & Decoupled Ring Boundaries

The codebase is structured into four decoupled architectural rings, ensuring HTTP routers only handle protocol concerns while delegating all business logic to application use cases via Dependency Injection:

| LAYER                | DIRECTORY               | IMPLEMENTED COMPONENTS & ARCHITECTURAL BOUNDARY                                                                                                                                                                                                                |
|----------------------|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Domain Layer         | backend/domain/         | Entities (ParsedTelemetry, RemediationRunbook, PreCheckCommand, RemediationCommand, PostCheckCommand, RollbackCommand, VendorDocCitation, AuditLedgerEntry), Value Objects, Domain Enums, and Exceptions.                                                      |
| Application Layer    | backend/application/    | Pure Use Cases (SynthesizeRemediationRunbookUseCase, IngestTelemetryBatchUseCase, QueryVendorSourcesUseCase), DTOs, and Abstract Port Interfaces (IVectorRepository, IAISynthesizer, IAuditRepository, ITelemetryParser, ICacheService).                       |
| Infrastructure Layer | backend/infrastructure/ | Concrete Adapters: AsyncpgVectorRepository (HNSW + BM25 RRF), InMemoryVectorRepository (air-gapped corpus), PgAuditRepository (JSONB audit ledger), DeterministicSynthesizer, ResilientLLMAdapter (tenacity retries), RegexTelemetryParser, RedisCacheService. |
| Presentation Layer   | backend/presentation/   | Dependency Injection container (dependencies.py), Thin REST Routers (troubleshoot_router, telemetry_router, health_router), and Real-Time WebSockets (telemetry_ws.py).                                                                                        |

### 3. Phase 4: FastAPI Presentation Routers, DI Container & WebSockets

Phase 4 connected the application use cases to HTTP and real-time WebSocket interfaces using a centralized Dependency Injection container (backend/presentation/dependencies.py):

| CONTROLLER / ROUTE    | HTTP / WS VERB | INJECTED USE CASE & OPERATIONAL SPECIFICATION                                                                                                        |
|-----------------------|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| /troubleshoot         | POST           | Injects SynthesizeRemediationRunbookUseCase. Analyzes raw syslog, executes hybrid RRF retrieval, synthesizes 4-stage runbook, and logs audit record. |
| /troubleshoot/sources | GET            | Injects QueryVendorSourcesUseCase. Queries pgvector/lexical index and returns vendor citations matching vendor/protocol filters.                     |
| /troubleshoot/audit   | GET            | Injects IAuditRepository. Retrieves permanent troubleshooting execution records with JSONB remediation & rollback details.                           |
| /telemetry/parse      | POST           | Injects ITelemetryParser. Parses raw syslog line and returns normalized vendor, event code, interface, and protocol domain entity.                   |
| /telemetry/ingest     | POST           | Injects IngestTelemetryBatchUseCase. Ingests log batches and conditionally triggers automated remediation for CRITICAL/ERROR events.                 |
| /health & /           | GET            | System health check probe returning database connectivity, vector search status, version 2.0.0, and API endpoint catalog.                            |
| /ws/telemetry         | WebSocket      | Managed by ConnectionManager. Streams live multi-vendor syslog events and broadcasts incident alerts to connected NOC consoles.                      |
| /ws/troubleshoot      | WebSocket      | Streams live step-by-step synthesis progress: parsing → retrieval → synthesizing → runbook_completed.                                                |

### 4. The 4-Stage Carrier Remediation Runbook Lifecycle

To prevent accidental carrier outages, VAT mandates a deterministic 4-stage execution sequence for all generated playbooks:

|                                                                                                              |                                                                                                       |                                                                                                            |                                                                                           |
|--------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| <b>01. PRE-CHECKS</b><br>Read-Only Inspection<br>Non-destructive state queries<br>validate baseline anomaly. | <b>02. REMEDIATION</b><br>Target CLI Config<br>Exact vendor commands with<br>configuration mode tags. | <b>03. POST-CHECKS</b><br>Convergence Test<br>Empirical validation to verify<br>packet recovery & peering. | <b>04. ROLLBACK</b><br>Safe Reversion<br>Immediate fail-safe undo if<br>validation fails. |
|--------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|

### 5. Quality Assurance & Automated Verification Results

**Automated Test Suite Execution:** pytest tests/ -v  
**Status:** 57 PASSED IN 5.01s (100% PASS RATE)

- **Presentation REST Controllers:** /troubleshoot, /sources, /audit, /parse, /ingest, /health • **PASSED**
- **Dependency Injection Overrides:** Runtime repository & use case mocking verification • **PASSED**
- **Real-Time WebSockets Streaming:** /ws/telemetry event stream & /ws/troubleshoot RAG synthesis progress • **PASSED**
- **Multi-Vendor Syslog Parsing:** Cisco BGP/OSPF, Juniper Junos RPD, VeloCloud SD-WAN, Arista MLAG • **PASSED**
- **Hybrid Vector Search & RRF:** Dense pgvector HNSW Cosine + Sparse BM25 tsvector Fusion • **PASSED**
- **Air-Gapped Offline Fallback:** In-memory corpus & 384-dimensional vector embedding normalization • **PASSED**
- **4-Stage Remediation Lifecycle:** Pre-Checks, Target CLI Config, Post-Checks, Rollback Playbook • **PASSED**

## 6. Phase 5 Roadmap: Modern NOC Frontend (Next.js / TypeScript)

With backend Clean Architecture and WebSockets verified, **Phase 5 (Frontend Component Architecture)** will build:

1. **Next.js (React) / TypeScript App Router:** Strict type-safety matching backend Pydantic models and DTOs.
2. **Obsidian Slate Dark Mode Design System:** High-density split-pane NOC canvas (zero emojis, JetBrains Mono CLI typography, glassmorphism, Framer Motion transitions).
3. **Zustand Incident Store:** Efficient live telemetry stream buffering and WebSocket subscription state.
4. **Interactive 4-Stage Runbook Canvas:** Color-coded blast radius indicators (LOW, MEDIUM, HIGH) with interactive CLI copy.